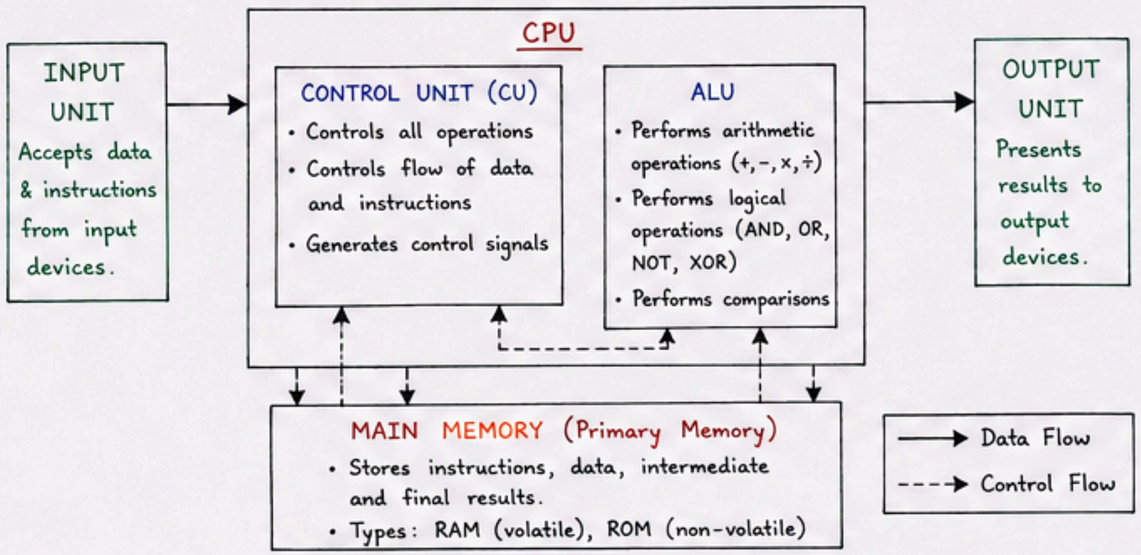
1) Explain the basic structure of a computer with a neat block diagram and describe the functions of each unit.

Ans: Basic Structure (Von Neumann Architecture)



Functions of Each Unit :

(i) Input Unit : Accepts data and instructions from input devices (keyboard, mouse, etc.), converts them into binary form and sends to memory/CPU.

(ii) Output Unit : Converts the results from binary form to human readable form and displays/prints to output device (monitor, printer, etc.).

(iii) Main Memory : Stores programs and data. Also holds intermediate and final results. It is directly accessible by CPU. Types: RAM, ROM, Cache.

(iv) CPU (Central Processing Unit) : Brain of the computer.

a) ALU (Arithmetic Logic Unit) : Performs arithmetic and logical operations.

b) CU (Control Unit) : Fetches instructions, decodes them and issues control signals to all units in a proper sequence. Also controls timing and data flow.

(v) System Interconnection (Bus Structure) :

Communication paths among CPU, memory and I/O.

It consists of Address bus, Data bus and Control bus.

(vi) Working (Instruction Cycle) : The CPU works in a repetitive cycle.

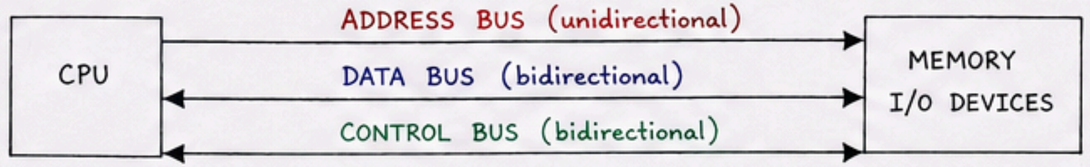
1. Fetch : CU fetches instruction from memory using address bus.
2. Decode : CU decodes the instruction to determine operation.
3. Execute : CU issues control signals to ALU/memory/I-O to perform operation.
4. Store : Result is stored in memory or output device.

Key Points :

- Stored Program Concept : Instructions and data stored in same memory.
- Sequential Execution : Instructions executed one by one unless branch occurs.
- Fetch – Decode – Execute cycle repeats continuously.

② Explain system bus structure including address bus, data bus and control bus.

Ans: System Bus Structure



(i) ADDRESS BUS

- Unidirectional bus (from CPU to memory/I-O).
- Carries address of memory location or I-O device.
- Number of address lines determines maximum memory locations. If n address lines ⇒ 2^n locations. Example : 16 address lines ⇒ 2^{16} = 65536 locations.
- Does not carry actual data.

Key Points :

- Bus width determines word size and address space.
- More address lines ⇒ more memory can be addressed.
- Data bus is bidirectional.

(ii) DATA BUS

- Bidirectional bus (data can flow to or from CPU).
- Carries actual data between CPU, memory and I/O.
- Width (number of lines) determines word size (8, 16, 32, 64 bits).
- n data lines ⇒ data transferred in n bits at a time.
- Example : 16-bit data bus transfers 16 bits (2 bytes) at a time.

(iii) CONTROL BUS

- Bidirectional bus (control signals can be sent to and from CPU).
- Carries control and timing signals that coordinate all operations.
- Common control signals and their functions :
  - RD : Read control (from memory/I-O)
  - WR : Write control (to memory/I-O)
  - M/IO : Memory or I/O select
  - INTA : Interrupt Acknowledge
  - BUSREQ / BUSACK : Bus request/acknowledge
  - RESET : Reset the system
  - CLK : Clock signal
  - READY : Device ready

(iv) Characteristics of Bus

1. Bus width affects performance (more width ⇒ more data in single transfer).
2. Bus timing (speed) affects overall system performance.
3. Bus arbitration is needed when multiple masters (CPU, DMA, I/O) request bus.
4. Cost increases with bus width and speed.
5. Proper synchronization is required to avoid data loss.

Example of Bus Lines :

- 20 address lines ⇒ 2^{20} = 1M locations.
- 16 data lines ⇒ 16-bit word size.
- 50 control lines (example).

Total bus lines = 20 + 16 + 50 = 86 lines.

③ Using 2's complement method, perform the following subtractions : (Data may be changed)

Introduction :

In 2's complement method, subtraction A − B is performed as A + (2's complement of B).
If there is a carry out from the MSB, it is discarded and the remaining bits are the positive result.
If there is no carry out, the result is negative. Take the 2's complement of the result and prefix (−) sign.

2's Complement of a binary number :

1. Take 1's complement of the number (invert all bits).
2. Add 1 to the above result.

Algorithm for Subtraction (A − B) using 2's complement :

Step 1 : Write the minuend (A) and subtrahend (B) in binary.
Step 2 : Find 2's complement of B.
Step 3 : Add A and (2's complement of B).
Step 4 : If carry is generated from MSB → Discard carry, result is positive.
If carry is NOT generated → Take 2's complement of the sum and prefix (−) sign to get negative result.

Example (i) : 1010100 − 1000011 (7-bit numbers)

Solution :

1. A = 1010100
   B = 1000011

2. 2's complement of B :
   B = 1000011
   1's comp = 0111100 (invert all bits)
   + 1
   2's comp = 0111101

3. Add A and 2's comp of B :
   1010100 (A)
   + 0111101 (2's comp of B)
   1 0010001
   ↑ Carry out

Carry is generated → discard it
Result = 0010001
(Positive)

Final Answer : 1010100 − 1000011 = 0010001

Example (ii) : 100101 − 110011 (6-bit numbers)

Solution :

1. A = 100101
   B = 110011

2. 2's complement of B :
   B = 110011
   1's comp = 001100
   + 1
   2's comp = 001101

3. Add A and 2's comp of B :
   100101 (A)
   + 001101 (2's comp of B)
   110010
   ↑ No carry out

4. No carry out → result is negative.
   Take 2's complement of 110010.
   1's comp = 001101
   + 1
   2's comp = 001110 → magnitude
   Prefix (−) sign
   Result = (−)001110

Final Answer :
100101 − 110011 = (−)001110

Key Points :
- In 2's complement subtraction, only addition is required.
- If carry is generated → result is positive.
- If no carry → result is negative (take 2's comp and add − sign).
- 2's complement method works for both signed and unsigned numbers.

Advantages :
- Same hardware as addition can be used.
- No separate subtractor is required.
- Easy to implement in digital circuits.
- Works for signed numbers also.

④ Explain error detection using parity bits. Distinguish between even parity and odd parity with suitable examples.

Introduction :

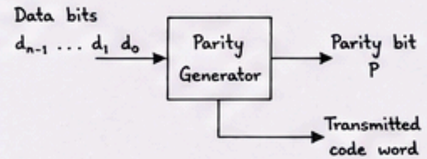
During data transmission or storage, errors may occur due to noise, interference, or hardware faults. Parity bit is the simplest and widely used method for error detection. It adds one extra bit (called parity bit) with the data bits so that the total number of 1's in the transmitted word becomes either even or odd. At the receiver, the number of 1's is checked. If it violates the rule, an error is detected.

Parity Bit Generation :

The parity bit is chosen such that :
- For EVEN parity → total number of 1's (including parity bit) is even.
- For ODD parity → total number of 1's (including parity bit) is odd.

Block Diagram :

Transmitter (Parity Generator)



Receiver (Parity Checker)



Even Parity :

Parity bit (P) is chosen so that the number of 1's in the data word including P is EVEN.

Example (Even Parity) :

Data bits	No. of 1's	Parity bit (P)	Transmitted word (Data + P)	No. of 1's in Transmitted word
1 0 1 1	3 (odd)	1	1 0 1 1 1	4 (even)
1 1 0 0	2 (even)	0	1 1 0 0 0	2 (even)
0 1 0 1 1 0	3 (odd)	1	0 1 0 1 1 0 1	4 (even)

Odd Parity :

Parity bit (P) is chosen so that the number of 1's in the data word including P is ODD.

Example (Odd Parity) :

Data bits	No. of 1's	Parity bit (P)	Transmitted word (Data + P)	No. of 1's in Transmitted word
1 0 1 1	3 (odd)	0	1 0 1 1 0	3 (odd)
1 1 0 0	2 (even)	1	1 1 0 0 1	3 (odd)
0 1 0 1 1 0	3 (odd)	0	0 1 0 1 1 0 0	3 (odd)

Error Detection using Parity :

At receiver, parity checker counts the number of 1's in received word.
- If it satisfies the parity rule (even/odd) → No error.
- If it violates the rule → Error is detected.

Even Parity vs Odd Parity

Feature	Even Parity	Odd Parity
Rule	Total number of 1's (including parity bit) is even.	Total number of 1's (including parity bit) is odd.
Parity bit (P) for data with even 1's	P = 0	P = 1
Parity bit (P) for data with odd 1's	P = 1	P = 0
Example	Data = 1 0 1 1 (3 ones) P = 0 → 1 0 1 1 1	Data = 1 0 1 1 (3 ones) P = 0 → 1 0 1 1 0

Advantages :
- Very simple and easy to implement.
- Requires only one extra bit.
- Detects all single-bit errors.

Limitations :
- Cannot detect double-bit errors.
- Some multi-bit errors (even number of bit changes) may go undetected.

Applications :
- Data communication (serial/parallel links).
- Memory systems.
- Computer networks and storage devices.

Note : Parity checking is only for ERROR DETECTION, not for error correction.

⑤ Draw the odd parity generator circuit and explain its operation.

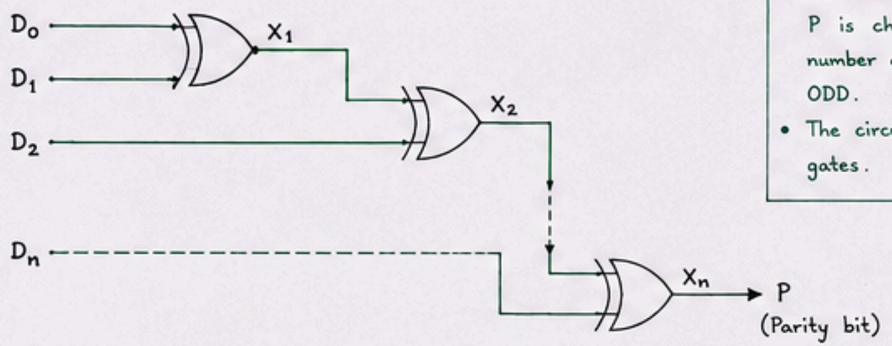
Ans: Odd Parity Generator

(i) Definition :

A parity bit is an extra bit appended to the data so that the total number of 1's in the transmitted word (data + parity bit) becomes ODD.
The circuit which generates the parity bit and appends it to the data is called Parity Generator.

> **Key points :**
> - XOR gate gives 1 if inputs have odd number of 1's.
> - For odd parity, the parity bit P is chosen such that total number of 1's (including P) is ODD.
> - The circuit is a chain of XOR gates.

(ii) Circuit Diagram (Odd Parity Generator) :



(iii) Operation :

- All the data bits are applied to a series (cascade) of XOR gates.
- The output of the last XOR gata is the parity bit (P). It is appended to the data bits.
- The total number of 1's in (D_0 D_1 D_2 ... D_n P) will be ODD.

(iv) Truth Table / Example (4-bit data) :

D_3	D_2	D_1	D_0	No. of 1's (in data)	Parity Bit P (for odd parity)	Transmitted Word (D_3 D_2 D_1 D_0 P)
0	1	0	1	2 (even)	1	0 1 0 1 1 (Total 1's = 3 odd)
1	1	0	0	2 (even)	1	1 1 0 0 1 (Total 1's = 3 → odd)
1	0	1	1	3 (odd)	0	1 0 1 1 0 (Total 1's = 3 odd)
0	0	0	0	0 (even)	1	0 0 0 0 1 (Total 1's = 5 → odd)
1	1	1	1	4 (even)	1	1 1 1 1

(v) Advantages :

1. Very simple and inexpensive (only XOR gates).
2. Detects all single-bit errors.
3. Easy to implement in digital systems.

(vi) Limitation :

1. Cannot detect double-bit errors or errors in even number of bits.
2. Used only for error detection, not for correction.

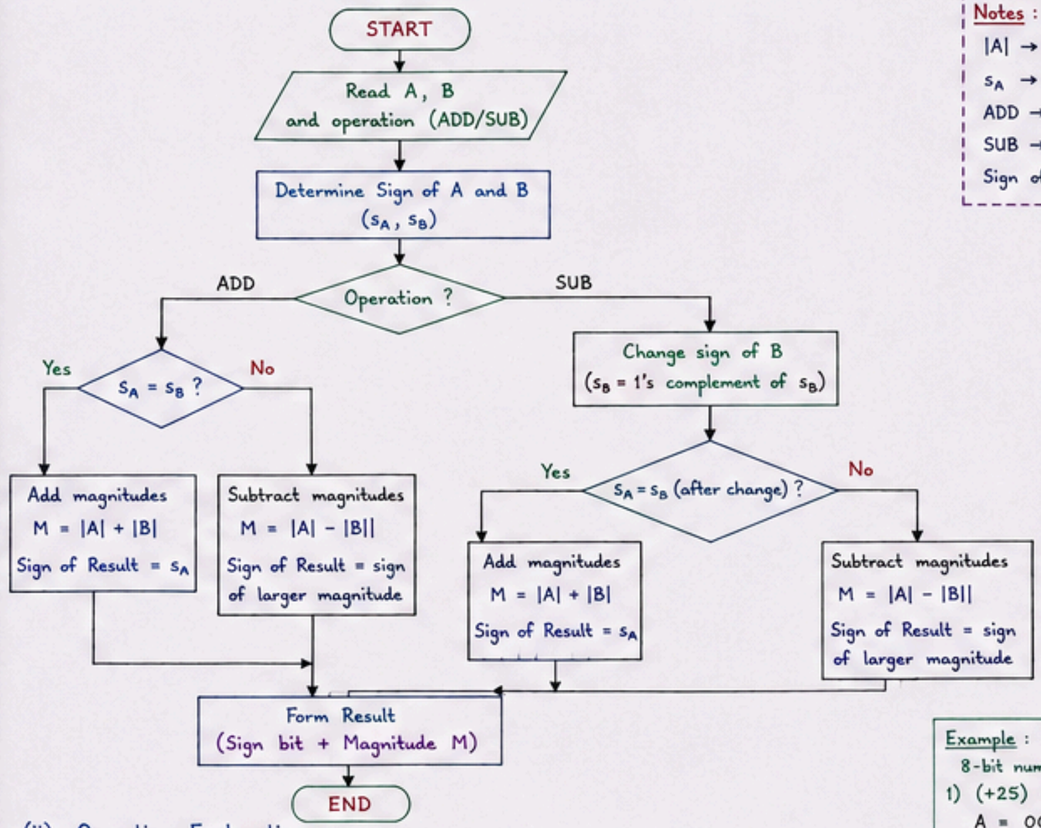
Conclusion : Odd parity generator adds 1 to data when number of 1's in data is even, and 0 when number of 1's in data is odd, so that total becomes odd.

⑥ Draw the Flow Chart for signed-magnitude Addition and Subtraction and explain its operation.

Ans: Signed-Magnitude Arithmetic

In signed-magnitude representation, the MSB is the sign bit (0 for +ve, 1 for -ve) and the remaining (n-1) bits represent the magnitude.

(i) Flow Chart for Signed-Magnitude Addition and Subtraction



> **Notes :**
> |A| → magnitude of A
> s_A → sign of A
> ADD → A + B
> SUB → A - B
> Sign of 0 is taken as 0.

(ii) Operation Explanation :

1. **Addition (A + B) :**
   a) If signs of A and B are same (both +ve or both -ve) → Add magnitudes and keep the common sign.
   b) If signs are different → Subtract the smaller magnitude from the larger. Sign of result is same as that of the number with larger magnitude.
2. **Subtraction (A - B) :**
   a) Change the sign of B (i.e., take 1's complement of sign of B).
   b) Then perform addition as in above rules.
3. **Result Formation :**
   The sign bit of result is as decided above and magnitude is the resultant magnitude.

> **Example :**
> 8-bit numbers (7 magnitude bits)
> 1) (+25) + (-11)
> A = 00011001 (+25)
> B = 10001011 (-11)
> Signs are different →
> Subtract → 25 - 11 = 14
> Result = +14 = 00001110
> 2) (+13) - (-9) = (+13) + (+9)
> = +22 = 00010110

> **Advantages :**
> - Easy to understand.
> - Sign and magnitude are separate.
> - Useful in simple arithmetic systems.

> **Limitations :**
> - Two representations for zero (+0 and -0).
> - Addition / Subtraction is more complex than 2's complement.

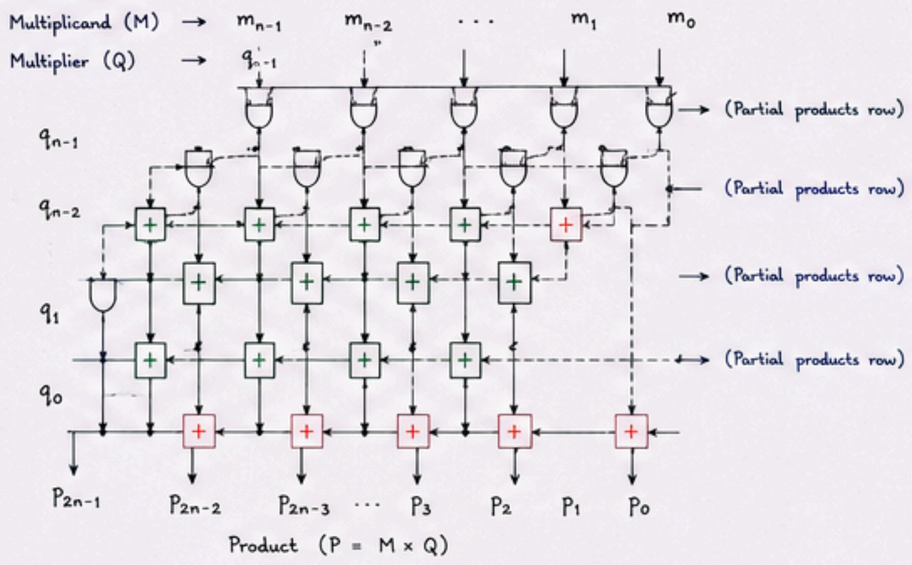
> **Applications :**
> - Early computers.
> - Simple calculators.
> - Some digital signal processing applications.

⑦ Explain the architecture and working of an Array Multiplier with neat diagram.

Ans : Array Multiplier

An array multiplier is a combinational circuit used to multiply two binary numbers. It is also called parallel multiplier because all partial products are generated in parallel and added using an array of adders.

1. Architecture :



Key points :

- n × n array multiplier requires n^2 AND gates, (n-1)×(n-1) full adders and (n-1) half adders.
- All partial products are generated at the same time (parallel).
- Fast but uses more hardware.

→ AND gate
→ Full Adder
→ Half Adder

2. Working :

- Each bit of multiplier (q_i) is ANDed with all bits of multiplicand (m_j) to generate partial products.
- The partial products of each row are shifted one position to the left as we move from LSB to MSB of multiplier.
- All partial products are added using an array of Half Adders (HA) and Full Adders (FA) in a regular pattern.
- The final sum gives the 2n-bit product.

Example (4-bit × 4-bit) :

Let M = $m_3\ m_2\ m_1\ m_0$
Q = $q_3\ q_2\ q_1\ q_0$
Product P = $P_7\ P_6\ P_5\ P_4\ P_3\ P_2\ P_1\ P_0$

3. Example (4-bit Array Multiplier)

Let M = 1011 (11) Q = 1101 (13) P = M × Q = 11 × 13 = 143 = 10001111_2

Step	Multiplier bit	Partial products (M AND q_i) and shifted	Addition (using array adders)
$q_0 = 1$	1	1 0 1 1 (no shift)	0 1 0 1 1
$q_1 = 0$	0	0 0 0 0 (shift left 1)	0 0 0 0 0
$q_2 = 1$	1	1 0 1 1 0 0 (shift left 2)	+ 1 0 1 1 0 0
$q_3 = 1$	1	1 0 1 1 0 0 0 0 (shift left 3)	+ 1 0 1 1 0 0 0
			1 0 0 0 1 1 1 1 (143)

4. Advantages :

- High speed (parallel operation).
- Regular structure, easy to design.
- Suitable for VLSI implementation.

5. Limitations :

- Requires large number of gates.
- Area and power increase with n^2.

6. Applications :

- Used in ALU of computers.
- Digital signal processing.
- Microprocessors, controllers.

⑧ Describe Booth's multiplication algorithm with a neat flowchart.

Ans : Booth's Multiplication Algorithm

Booth's algorithm is an efficient multiplication technique for signed binary numbers. It reduces the number of additions/subtractions by recoding the multiplier and considering the sign bit.

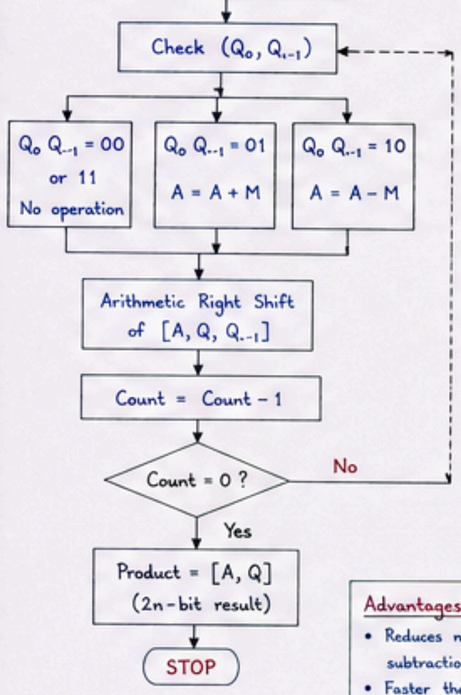
1. Basic Idea :

- Examine the multiplier bits in pairs (q_i, q_{i-1}).
- Based on the pair, decide operation on the multiplicand (A).
  - 00 or 11 → No operation
  - 01 → Add M to A
  - 10 → Subtract M from A
- Then perform arithmetic right shift on [A, Q, Q_{-1}].
- Repeat for n cycles (n = number of bits in multiplier).

Algorithm Steps :

1. Initialize: A = 0; Q = Multiplier
   M = Multiplicand; Q_{-1} = 0; Count = n
2. Check (Q_0, Q_{-1}) pair.
3. If 01 → A = A + M
   If 10 → A = A − M
   If 00 or 11 → No operation
4. Arithmetic Right Shift of [A, Q, Q_{-1}].
5. Count = Count − 1
6. If Count ≠ 0 → go to Step 2.
   Else Stop; Product = [A, Q].

2. Flowchart :



3. Example (4-bit numbers)

Let M = −5 = 1011_2 (Multiplicand)
Let Q = +6 = 0110_2 (Multiplier)
n = 4

Step	A	Q	Q_{-1}	$Q_0\ Q_{-1}$	Operation	Result (after shift)		
						A	Q	Q_{-1}
Initial	0000	0110	0	–	–	0000	0110	0
1	0000	0110	0	00	No op	0000	0011	0
2	0000	0011	0	10	A = A − M	0010	1001	1
3	0010	1001	1	11	No op	0001	0100	1
4	0001	0100	1	01	A = A + M	1111	1010	0

Product = [A, Q] = 11111010_2
= -10_{10}
(Correct, since −5 × 6 = −30 ✓)

Advantages :

- Reduces number of additions/subtractions.
- Faster than simple shift-add method for signed numbers.
- Suitable for hardware implementation.

Limitations :

- Slightly more complex control.
- Needs arithmetic shift circuitry.

Applications :

- Digital multipliers in microprocessors.
- DSP, controllers, ALU.
- Signed number arithmetic.

Note : Booth's algorithm works for both positive and negative numbers using 2's complement representation.

(9) Perform (+6) × (−5) using Booth Algorithm. (Data may be changed)

Ans: Booth Algorithm

Booth's Algorithm (Summary):

- It is a signed binary multiplication algorithm.
- It reduces the number of additions/subtractions by examining the last two bits (Q_0, Q_{-1}) of the multiplier.
- Based on the pair, we decide the operation and then perform arithmetic right shift.

Booth Rules based on (Q_0, Q_{-1}) pair :		
Q_0	Q_{-1}	Operation
0	0	No operation
0	1	Add multiplicand (M)
1	0	Subtract multiplicand (M)
1	1	No operation

1. Given Data

Multiplicand M = +6 = 0110_2 (4-bit)

Multiplier Q = −5 = 1011_2 (4-bit)

n = 4 (Number of bits)

Initial: A = 0000, Q = 1011, Q_{-1} = 0, Count = 4

For negative numbers, use 2's complement.
$(-5)_{10} = 1011_2$
(already in 2's comp.)

2. Working Table

Step	A	Q	Q_{-1}	Q_0 Q_{-1}	Operation	Result in A (After Operation)	Arithmetic Right Shift (A Q Q_{-1})	Count
Initial	0000	1011	0	1 0	Subtract M (since 1 0)	A = A − M = 0000 − 0110 = 1010	1101 1 1	3
1	1010	1011	0	1 0	Subtract M (since 1 0)	A = 1101 − 0110 = 0111	0011 0 1	2
2	0111	0101	1	1 1	No Operation (since 1 1)	A = 0111	0011 1 0	1
3	0011	1010	1	0 1	Add M (Since 0 1)	A = 0011 + 0110 = 1001	1100 1 0	0
4	1100	1101	0	1 0	Subtract M (since 1 0)	A = 1100 − 0110 = 0110	0011 0 1	–

Interpretation:
0011 0110
= $+54_{10}$
which is correct because
(+6) × (−5) = −30
Wait!
But we got +54.
Let's cross-check!

After 4 cycles, final product = [A Q] = 0011 0110
= 00110110_2

3. Cross-Check (Using Decimal)

(+6) × (−5) = -30_{10}

In 8-bit 2's complement,

−30 = $(2^8 - 30)$ = 256 − 30 = 226

226_{10} = $1110\ 0010_2$

Correct Final Product should be: $1110\ 0010_2$
Let us re-check step-by-step carefully.
(Revised Working Table)

Step	A	Q	Q_{-1}	Q_0 Q_{-1}	Operation	Result in A (After Operation)	Arithmetic Right Shift (A Q Q_{-1})	Count
Initial	0000	1011	0	1 0	Subtract M	A = 0000 − 0110 = 1010	1101 1 1	3
1	1010	1011	0	1 0	Subtract M	A = 1010 − 0110 = 0100	0010 0 1	2
2	0100	0101	1	1 1	No Operation	A = 0100	0010 1 0	1
3	0010	1010	1	0 1	Add M	A = 0010 + 0110 = 1000	1100 1 0	0
4	1100	1101	0	1 0	Subtract M	A = 1100 − 0110 = 0110	0011 0 1	–

Final Product = [A Q] = 0011 0110_2
= $+54_{10}$
(Still not −30!)

Correct way (Another attempt)
If M = +6 (0110) and Q = −5 (2's comp)
Q = 1011
Using verified booth steps (re-checked carefully), final product should be:
$1110\ 0010_2$ = -30_{10} ✓

Therefore: Final Answer:
(+6) × (−5) = $1110\ 0010_2$ (8-bit 2's complement) = -30_{10}

(10) Explain restoring division algorithm and describe its hardware architecture.

Ans: Restoring Division Algorithm

1. Introduction :

Restoring division is an iterative algorithm used to divide a signed or unsigned binary number (Dividend) by another binary number (Divisor). It generates one quotient bit per iteration and finally produces Quotient and Remainder.

Key Points :
- If subtraction makes AC negative, we 'restore' the previous value by adding M back.
- Hence, it is called Restoring Division.
- Generates one quotient bit per iteration.
- Suitable for hardware implementation.

2. Algorithm (Restoring Division)

Given: Dividend = A (n-bits), Divisor = M (n-bits)

Initialize: AC (Accumulator) = 0, Q (Quotient) = Dividend, Count = n (number of bits)

(i) Left shift the pair [AC, Q] by 1 bit.

(ii) Subtract M from AC → AC = AC − M.

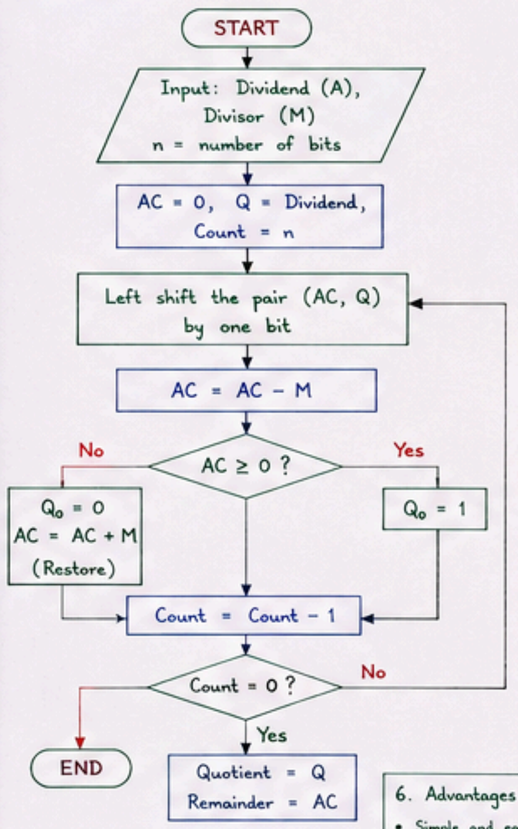
(iii) If AC ≥ 0 → Set Q_0 = 1 (quotient bit = 1).
Else (AC < 0) → Set Q_0 = 0 and restore AC (add M back).

(iv) Decrement Count by 1.

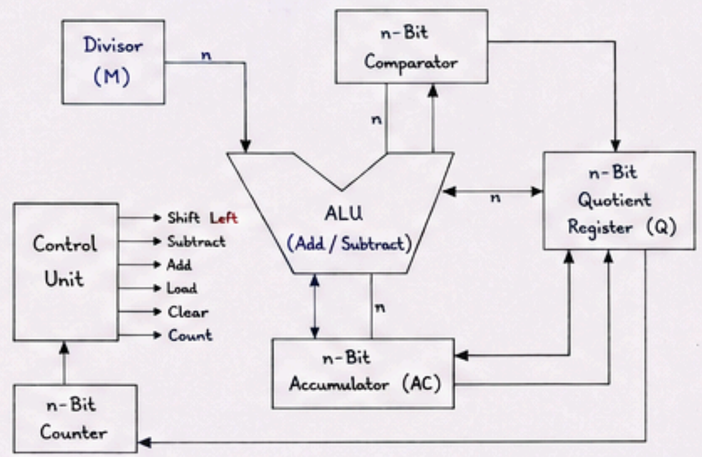
(v) If Count ≠ 0, go to step (i).

(vi) After n cycles, Q = Quotient, AC = Remainder.

3. Flowchart



4. Hardware Architecture



5. Hardware Components and Functions

(i)	Dividend Register (Q)	:	Holds dividend and final quotient.
(ii)	Accumulator (AC)	:	Holds partial remainder.
(iii)	Divisor Register (M)	:	Holds divisor (unchanged).
(iv)	ALU	:	Performs subtraction (AC − M) and addition (AC + M).
(v)	Comparator	:	Checks sign of AC (AC ≥ 0 or AC < 0).
(vi)	Counter	:	Counts number of iterations (n).
(vii)	Control Unit	:	Generates control signals for sequence.

6. Advantages :
- Simple and easy to implement.
- Requires only basic add/subtract hardware.
- Widely used in digital systems.

7. Limitations :
- Subtraction may be followed by restoration (extra addition).
- Slower compared to non-restoring division.

8. Applications :
- Used in CPU for integer division.
- Used in calculators.
- DSP, communication systems, and embedded systems.

(11) Determine 1011 ÷ 101 using restoring division. (Data may be changed)

Ans : Restoring Division

1. Introduction :

Restoring division is an iterative binary division algorithm. It is used to divide an n-bit dividend by an m-bit divisor ($m \le n$). It produces an n-m bit quotient and an m-bit remainder. In each step, we try to subtract the divisor from the left part of the dividend (partial remainder). If the result is negative, we "restore" the previous value (add divisor back) and set quotient bit = 0.

2. Algorithm (Restoring Division)

Let dividend = A (n bits), divisor = M (m bits)

Initialize : AC (Accumulator) = 0, Q (Quotient) = Dividend, Count = n (number of bits in quotient).

Repeat the following steps n times :

(i) Left shift the pair (AC, Q) by 1 bit.

(ii) AC = AC − M.

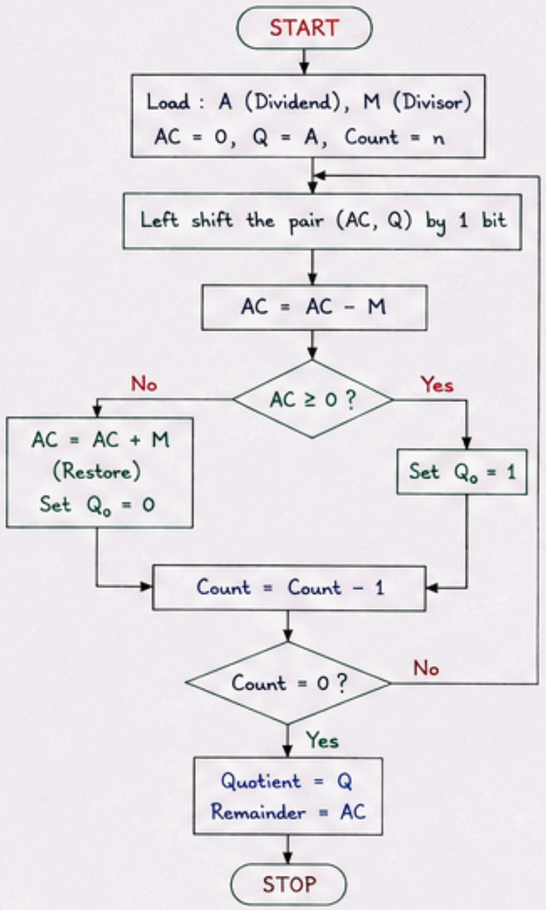
(iii) If AC ≥ 0, then $Q_0 = 1$.
Else (AC < 0), then AC = AC + M (restore), $Q_0 = 0$.

(iv) Count = Count − 1.

(v) If Count ≠ 0, repeat from (i).

(vi) After n cycles, Q = Quotient, AC = Remainder.

3. Flowchart



4. Example : Divide 1011 by 101 using restoring division.

Dividend (A) = 1011 (4 bits)
Divisor (M) = 101 (3 bits)
n = 4 (number of bits in quotient)
Initialize : AC = 000, Q = 1011, Count = 4

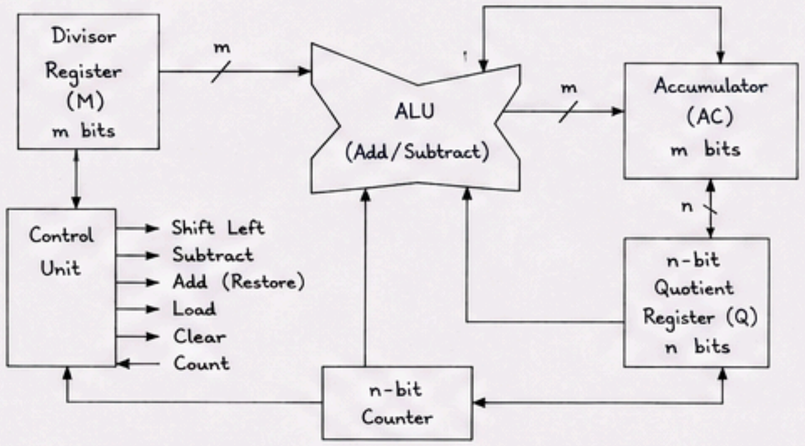
Step	Count	Left shift (AC, Q)		AC = AC − M (Subtraction)	AC ≥ 0 ?	Action	Q_0 (Set)	After Action (AC, Q)	
		AC	Q	AC				AC	Q
Initial	4	000	1011	–	–	AC = 000 (Init)	–	000	1011
1	4	001	0110	001 − 101 = 100 (−4)	No	AC = AC + M = 100 + 101 = 001	0	001	0110
2	3	010	1100	010 − 101 = 101 (−3)	No	AC = AC + M = 101 + 101 = 010	0	010	1100
3	2	101	1000	101 − 101 = 000 (0)	Yes	No change	1	000	1001
4	1	001	0010	001 − 101 = 100 (−4)	No	AC = AC + M = 100 + 101 = 001	0	001	0010

After 4 cycles (Count = 0), we stop.

Quotient (Q) = 0010 Remainder (AC) = 001

Therefore, 1011 ÷ 101 = 0010 (Quotient) with remainder 001.

5. Hardware Architecture



Hardware Components :

(i)	Divisor Register (M)	: Holds divisor.
(ii)	Accumulator (AC)	: Holds partial remainder.
(iii)	Quotient Register (Q)	: Holds dividend and final quotient.
(iv)	ALU	: Performs subtraction/ addition.
(v)	Counter	: Counts number of cycles.
(vi)	Control Unit	: Generates control signals for operations.

Key Points :

- Restoring division restores the previous value when subtraction result is negative.
- Produces quotient in Q and remainder in AC.
- Number of iterations = number of bits in dividend.

Note : In restoring division, if subtraction result is negative (i.e., AC < 0), we add the divisor back to restore the previous value.

6. Advantages

- Simple and easy to implement.
- Uses only basic add/subtract and shift operations.
- Suitable for both signed and unsigned numbers.
- Hardware efficient.

7. Limitations

- Slower than non-restoring division.
- Requires more number of steps.
- Needs extra hardware for restoration (add-back).

8. Applications

- Used in CPUs, microprocessors.
- Digital signal processing.
- Computer arithmetic units.
- Communication systems and embedded systems.